Module 1

Architecture & Design Patterns

MVVM vs MVI

MVVM

- Multiple `StateFlow`s — one per state field
- View calls named ViewModel functions
- Side effects bolted on via `SharedFlow`
- Flexible, easy to adopt incrementally
- State updates are not atomic — inconsistency window between assignments

MVI

- Single immutable `UiState` object
- All interactions via `processIntent(Intent)`
- Side effects are a first-class `Channel<SideEffect>`
- Strict unidirectional data flow
- State updates are atomic — view never sees a half-updated state

MVVM and State Management

MVVM with `ViewModel` and `StateFlow` is the backbone of modern Android development. The ViewModel survives configuration changes and exposes UI state as a stream that composables or views observe. When combined with Unidirectional Data Flow — where events flow up and state flows down — the result is a UI that is predictable, testable, and easy to reason about. In Nami, every screen is driven by a single `UiState` data class emitted from a `StateFlow`, making the full screen state inspectable at any point.

Clean Architecture

- **Presentation** — UI layer: Composables, ViewModels, UI state
- **Domain** — Business logic: Use cases, entities, repository interfaces
- **Data** — Infrastructure: Repository implementations, network, database
- Dependencies only point **inward** — outer layers depend on inner, never the reverse
- Domain layer has **zero Android dependencies** — plain Kotlin, fully unit-testable

Use Cases

- Encapsulate a **single business operation** (e.g. `GetUserProfileUseCase`)
- Called by the ViewModel — keep ViewModels thin and focused on UI state
- Take repository interfaces as constructor parameters — easy to mock in tests
- One class, one responsibility — compose multiple use cases for complex flows

```
class LogoutUseCase @Inject constructor(
    private val userRepository: UserRepository,
    private val resetWishlists: ResetWishlists,
    private val clearBag: ClearBag,
    // ... more dependencies
) : Logout {
    override suspend operator fun invoke(): Result<Boolean, Unit> {
        return setPushNotificationsPreference(false).also {
            clearAllSources()
        }
    }

    private suspend fun clearAllSources() {
        resetWishlists()
        userRepository.logout()
        clearBag()
        // ... clear remaining sources
    }
}
```

Dependency Injection

- Classes declare **what they need** — not how to create it
- Dependencies are provided from the outside, making classes easy to test and replace
- **Hilt** is the standard DI framework on Android, built on top of Dagger
- `@Inject constructor` marks a class for automatic injection — no manual wiring
- `@HiltViewModel` connects the DI graph to the ViewModel lifecycle
- Scopes control lifetime: `@Singleton`, `@ActivityRetainedScoped`, `@ViewModelScoped`

DI Strategies Compared

Choose your DI approach based on app size and how early you want to catch errors — **Manual** for tiny apps, **Hilt** for production scale with compile-time safety, **Koin** for fast iteration with a clean DSL.

Factor	Manual	Hilt / Dagger	Koin
Build speed	Fast	Slow (KSP)	Fast
Error detection	Manual	Compile-time	Runtime
Learning curve	Low	High	Low
Scalability	Poor	Excellent	Moderate
KMP support	Yes	Partial	Yes
Boilerplate	High	Medium	Low

Modularization

- Split the app into independent Gradle modules with clear boundaries
- **Feature modules** own a vertical slice: UI, domain logic, and data for one feature
- **Core modules** share infrastructure: networking, database, design system, utils
- Modules depend on **interfaces, not implementations** — enforced by the build graph
- Parallel compilation: Gradle only rebuilds modules whose inputs changed
- Enables **dynamic delivery** — feature modules can be downloaded on demand